Data Management Strategy:

The platform receives high velocity real-time gaming telemetry combined with ultra-sensitive compliance and security information. We use a polyglot persistence, tiered architecture, and system design to meet the demands of regulatory compliance, low latency, and absolute auditability.

1. Database Architecture & Design Choices

For our system we have the two core persistent/storage components being used that suited the different needs of each: and real-time, transient, in-memory buffers for individual stream processing:

Limited Shoe-Order Store (High-Security Sequence Store):

Purpose: Consume and validate verified card-order sequences from the approved card-reading shoe.
Design Choice: An extremely limited, low latency relational or specialized cryptographic append-only store.
Card Integrity Card data should never leave the table and the table area. Encoded shoe data with the clear sequence data removed allows for instant and secure comparison without jeopardizing the physical cards at play.

Evidence + Audit Database (Long-Term Compliance Store):

Purpose: Storage: Collection of aggregate events, video references, audit logs, supervisor decisions/reviews.
A secure enterprise quality relational database (e.g. PostgreSQL with tight row-level security, & write-once-read-many/immutable archiving principals).
Why: Compliance regulators demand unaltered historical records of all gameplay, warnings and player/agent interactions. This data store separates compliance logging from game table processing in order to give investigations and audits a reliable, searchable record of truth with no effect on in-play game table performance.

2. Logical Data Splitting Approach

Data is physically segregated along operational boundaries to ensure security, privacy and performance:
Operational / Real-Time Tier (Transient) – within Processing block (Event collector, Round reconstruction, Detection engine). Session data and public card counts are strictly anonymous and transient (or maintained in-memory data stores such as Redis) in order to preserve player anonymity and ensure sub-millisecond detection.
Security Tier (Restricted Store) Sensitive physical hardware telemetry (shoe order data) accessed via isolating controls.
Audit Tier (Evidence Database): Contains past, cross-referenced evidence packages connecting video references, automated alerts, and reviewed/approved by staff decisions.

3. Other Alternatives Evaluated

During the development of this architecture we tested two high-level options:

Alternative 1: Monolithic Single Database Architecture

Concept: Piping data-all data: from real-time card shoes, event collectors, video references, and staff reviews-into one enormous relational database instance.
Tradeoffs: It is easier to initially deploy a single database, but it becomes a bottleneck. High-volume table telemetry and video reference index would compete with compliance queries creating latency spikes during live play. It also violates security best practices by combining high security hardware shoe data into general review logs in a single schema.

Alternative 2: Pure NoSQL / Document Store (MongoDB / DynamoDB)

Design: Write all event logs, audit trails to a distributed NoSQL document store.
Tradeoffs: Document stores excel at high write-scalability and flexible schemas for fast logging of events. But relational consistency (foreign-key integrity between evidence packages and staff reviews) and complex audit queries are more challenging to deliver naturally than with an actually-structured SQL audit store.